

RELATÓRIO DO LABORATÓRIO: Tempo e estado global

Universidade Federal do Ceará - Campus de Quixadá

Equipe: Mônica Maria Rodrigues de Castro

Maryana Moraes Sousa

Disciplina: Sistemas Distribuídos

Professor: Rafael Braga

1. Introdução

Em sistemas distribuídos, cada processo possui seu próprio relógio local, não existindo, em geral, um relógio global perfeitamente sincronizado entre todos os nós do sistema. Essa ausência de sincronização torna difícil determinar a ordem correta dos eventos distribuídos, o que pode comprometer a consistência do sistema.

Dessa forma, diversos mecanismos foram propostos para lidar com o problema do tempo em sistemas distribuídos, entre eles os relógios físicos, os relógios lógicos de Lamport e os relógios vetoriais. Este laboratório tem como objetivo implementar e analisar essas abordagens, comparando seus comportamentos e limitações por meio de simulações práticas em linguagem C++.

2. Objetivos

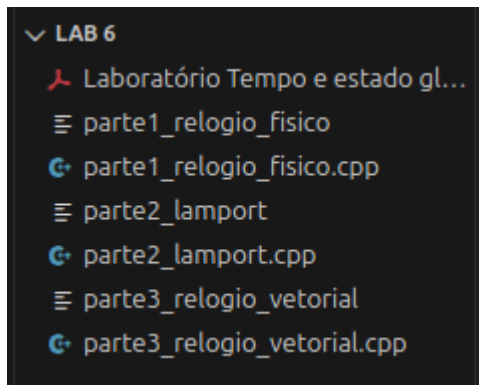
- Demonstrar as limitações do uso de relógios físicos em sistemas distribuídos.
- Implementar o algoritmo de relógio lógico de Lamport.
- Implementar relógios vetoriais para controle de causalidade e concorrência.
- Comparar os resultados obtidos em cada abordagem.

3. Tecnologias e Ferramentas Utilizadas

- Linguagem de programação: **C++**

- Biblioteca de threads: `std::thread`
- Controle de concorrência: `std::mutex`
- Controle de tempo: `std::chrono`
- Sistema Operacional: **Linux (Ubuntu)**
- Compilador: **g++ (GNU Compiler Collection)**

4. Estrutura do Projeto



5. Metodologia

O laboratório foi dividido em três partes independentes. Cada parte foi implementada como um programa separado em C++, contendo sua própria função `main()`. Para simular processos distribuídos, foram utilizadas threads, representando processos independentes que se comunicam entre si.

A latência de comunicação foi simulada por meio de chamadas à função `sleep_for`, introduzindo atrasos aleatórios. Para evitar conflitos na escrita simultânea no terminal, foi utilizado um mutex global, garantindo exclusão mútua durante a impressão dos logs.

6. Parte 1 – Relógio Físico

6.1 Descrição

Nesta etapa, foram criados três processos simulados que trocam mensagens entre si. Cada processo mantém um relógio físico local, obtido por meio da função

`std::chrono::system_clock::now()`). Todos os eventos de envio e recebimento de mensagens são registrados juntamente com seus respectivos timestamps físicos.

6.2 Objetivo

Demonstrar que o uso exclusivo de relógios físicos não garante uma ordenação correta dos eventos distribuídos, devido a atrasos de comunicação e ausência de sincronização global.

6.3 Observações

```
maryana-moraes@maryana-moraes-Aspire-A515-57:~/Documentos/UFC/8_Semestre/Sistemas Distribuidos/atividades/Laboratórios/Lab 6$ ./parte1_relogio_fisico
P2 ENVIU para P0 | Tempo fisico = 1767709932328
P0 RECEBEU de P2 | Tempo local = 1767709932328 | Tempo da msg = 1767709932328
P1 ENVIU para P2 | Tempo fisico = 1767709932865
P2 RECEBEU de P1 | Tempo local = 1767709932865 | Tempo da msg = 1767709932865
P0 ENVIU para P1 | Tempo fisico = 1767709932888
P1 RECEBEU de P0 | Tempo local = 1767709932888 | Tempo da msg = 1767709932888
```

Durante a execução, foi possível observar que, embora em alguns casos os tempos pareçam coerentes, essa ordem não é garantida. Em cenários mais complexos, eventos podem aparentar ocorrer fora de ordem, evidenciando a limitação dos relógios físicos em sistemas distribuídos.

7. Parte 2 – Relógio Lógico de Lamport

7.1 Descrição

Na segunda parte, cada processo mantém um contador lógico L , inicializado com valor zero. O funcionamento do relógio lógico de Lamport segue as seguintes regras:

- A cada evento local, o contador é incrementado ($L = L + 1$).
- Ao enviar uma mensagem, o processo envia o valor atual do contador lógico.
- Ao receber uma mensagem, o processo atualiza seu contador da seguinte forma:
$$L = \max(L_{\text{local}}, L_{\text{recebido)}} + 1.$$

7.2 Objetivo

Garantir que a ordem causal dos eventos seja preservada, independentemente dos atrasos de comunicação ou da falta de sincronização dos relógios físicos.

7.3 Observações

```
maryana-moraes@maryana-moraes-Aspire-A515-57:~/Documentos/UFC/8_Semestre/Sistemas Distribuidos/atividades/laboratórios/Lab 6$ ./parte2_lamport
P1 EVENTO: Enviando mensagem | L = 1
P2 RECEBEU de P1 | L = 2
P2 EVENTO: Enviando mensagem | L = 3
P0 RECEBEU de P2 | L = 4
P0 EVENTO: Enviando mensagem | L = 5
P1 RECEBEU de P0 | L = 6
```

Os resultados mostraram que os valores do relógio lógico são sempre crescentes e respeitam a causalidade entre eventos. No entanto, esse método não permite identificar se dois eventos ocorreram de forma concorrente.

8. Parte 3 – Relógios Vetoriais

8.1 Descrição

Na terceira etapa, cada processo mantém um vetor de tamanho N, onde N representa o número total de processos. Cada posição do vetor indica o conhecimento do processo sobre os eventos ocorridos em cada processo do sistema.

As regras utilizadas foram:

- Evento local: incrementa a posição correspondente ao próprio processo.
- Envio de mensagem: envia uma cópia do vetor.
- Recebimento de mensagem:
 - Cada posição do vetor é atualizada com o valor máximo entre o vetor local e o vetor recebido.
 - A posição correspondente ao processo receptor é incrementada.

8.2 Objetivo

Permitir a identificação precisa da causalidade entre eventos, além de possibilitar a detecção de eventos concorrentes.

8.3 Observações

```
maryana-moraes@maryana-moraes-Aspire-A515-57:~/Documentos/UFC/8_Semestre/Sistemas Distribuidos/atividades/laboratórios/Lab 6$ ./parte3_relogio_vetorial
P0 EVENTO: Enviando mensagem | V = [ 1 0 0 ]
P1 RECEBEU de P0 | V = [ 1 1 0 ]
P2 EVENTO: Enviando mensagem | V = [ 0 0 1 ]
P0 RECEBEU de P2 | V = [ 2 0 1 ]
P1 EVENTO: Enviando mensagem | V = [ 1 2 0 ]
P2 RECEBEU de P1 | V = [ 1 2 2 ]
```

Os relógios vetoriais fornecem uma visão mais completa do estado global do sistema, pois permitem distinguir eventos causais de eventos concorrentes, algo que não é possível utilizando apenas o relógio lógico de Lamport.

9. Como Compilar e Executar

Cada parte do laboratório foi implementada em um arquivo separado:

- parte1_relogio_fisico.cpp
- parte2_lamport.cpp
- parte3_relogio_vetorial.cpp

9.1 Compilação

No terminal, execute os seguintes comandos:

```
$ g++ -std=c++11 parte1_relogio_fisico.cpp -o parte1_relogio_fisico
$ g++ -std=c++11 parte2_lamport.cpp -o parte2_lamport
$ g++ -std=c++11 parte3_relogio_vetorial.cpp -o parte3_relogio_vetor
```

- g++ -std=c++11 parte1_relogio_fisico.cpp -o parte1
- g++ -std=c++11 parte2_lamport.cpp -o parte2
- g++ -std=c++11 parte3_relogio_vetorial.cpp -o parte3

9.2 Execução

./parte1

Obs: Um processo pode **receber uma mensagem com timestamp maior que o seu**.
Ordem aparente **não reflete causalidade**

./parte2

Obs: Valores de L sempre crescem
Ordem lógica respeita causalidade
Não detecta concorrência

./parte3

Obs: Vetores diferentes podem ser incomparáveis
Permite identificar eventos concorrentes

10. Conclusão

A realização deste laboratório permitiu compreender as limitações do uso de relógios físicos em sistemas distribuídos e a importância de mecanismos lógicos para garantir a consistência temporal. O relógio lógico de Lamport mostrou-se eficaz na preservação da ordem causal dos eventos, enquanto os relógios vetoriais forneceram uma solução mais completa ao possibilitar a identificação de eventos concorrentes.

Conclui-se, portanto, que mecanismos lógicos são essenciais para o correto funcionamento e análise de sistemas distribuídos.